



Università
di Catania

UNIVERSITY OF CATANIA

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

Giulio Pedicone

Cloud Registry

PROGETTO DI SISTEMI CLOUD

Anno Accademico 2024 - 2025

Indice

1	Introduzione al Progetto	2
1.1	Contesto e Motivazioni	2
1.2	Obiettivi del Progetto	2
1.3	Architettura del Sistema	3
1.4	Tecnologie e Servizi AWS Utilizzati	4
1.5	Metodologia DevOps e Infrastructure as Code	5
1.5.1	Infrastructure as Code (IaC) con Terraform	5
1.5.2	CI/CD con GitHub Actions	5
2	Demo del progetto	6
2.1	Accesso e Registrazione	6
2.2	Interfaccia Docente	9
2.2.1	Assegnazione Voti e Notifiche	9
2.2.2	Gestione Note Disciplinari (Container)	10
2.3	Interfaccia Studente	12
3	Dettagli Implementativi e Logica del Codice	13
3.1	Automazione avanzata con Terraform	13
3.2	Logica serverless e privacy delle notifiche	14
3.3	Integrazione cloud nel microservizio containerizzato	15
4	Automazione e Pipeline CI/CD	16
4.1	Struttura del Workflow	16
4.1.1	Preparazione dell'Ambiente e Gestione dello Stato	16
4.2	Deployment dell'Infrastruttura (IaC)	17
4.3	Delivery del Microservizio Containerizzato	17
4.4	Gestione della Sicurezza e dei Segreti	18
5	Conclusioni e Sviluppi Futuri	19

Capitolo 1

Introduzione al Progetto

1.1 Contesto e Motivazioni

La digitalizzazione del sistema scolastico richiede infrastrutture sempre più robuste, scalabili e ad alta disponibilità. Le soluzioni tradizionali *on-premise* per la gestione dei registri elettronici soffrono spesso di limitazioni legate alla scalabilità durante i picchi di carico e costi di manutenzione elevati.

Il progetto **CloudRegistry** nasce con l'obiettivo di superare queste limitazioni attraverso l'adozione di un approccio *Cloud-Native*. Sfruttando i servizi gestiti di Amazon Web Services (AWS), è stato possibile realizzare un'applicazione web moderna, sicura e resiliente, capace di adattarsi dinamicamente al carico di lavoro e di ridurre drasticamente l'overhead di gestione infrastrutturale.

1.2 Obiettivi del Progetto

L'obiettivo principale è la realizzazione di una piattaforma per la gestione scolastica che implementi le funzionalità core di un registro elettronico, garantendo al contempo sicurezza dei dati e separazione dei privilegi.

Le funzionalità chiave implementate includono:

- **Gestione Identità e Accessi (IAM):** Autenticazione sicura e distinzione granulare dei ruoli tra Docenti (permessi di scrittura) e Studenti (permessi di sola lettura).
- **Gestione Voti (Serverless):** Funzionalità CRUD (*Create, Read, Update, Delete*) per l'assegnazione e la consultazione dei voti scolastici, con calcolo automatico delle medie.
- **Note Disciplinari (Microservizi):** Un sistema dedicato per la gestione delle note disciplinari, implementato tramite architettura a container.
- **Notifiche Real-Time:** Sistema di notifica automatico via email (pattern *Pub/Sub*) per informare gli studenti in tempo reale delle nuove valutazioni.

1.3 Architettura del Sistema

L'architettura di CloudRegistry è di tipo **ibrido Cloud-Native**, combinando la flessibilità del paradigma *Serverless* con la portabilità dei *Microservizi Containerizzati*. Questa scelta progettuale permette di dimostrare la capacità di orchestrare diverse tipologie di risorse di calcolo all'interno dello stesso ecosistema.

L'infrastruttura è completamente disaccoppiata: il frontend (Single Page Application) comunica con il backend esclusivamente tramite API REST esposte tramite API Gateway.

CloudRegistry (Registro Elettronico) - Schema Architetture

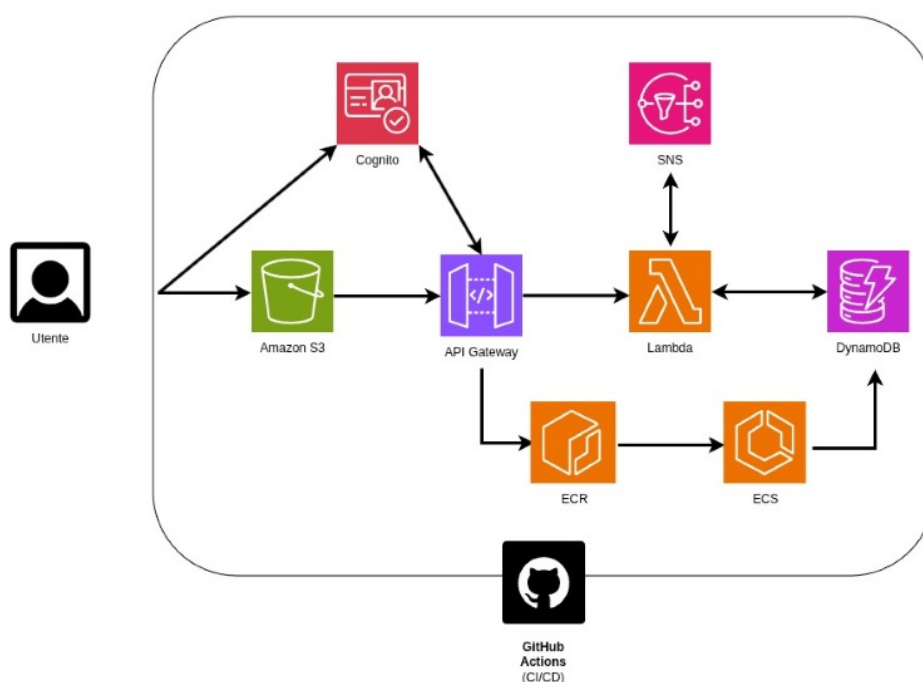


Figura 1.1: Diagramma architetturale ad alto livello di CloudRegistry su AWS.

1.4 Tecnologie e Servizi AWS Utilizzati

Il progetto fa uso intensivo dello stack AWS. Di seguito vengono dettagliati i servizi impiegati e il loro ruolo nell'architettura:

Amazon S3 (Simple Storage Service): Utilizzato per l'hosting statico del frontend (HTML, CSS, JavaScript), garantendo alta disponibilità e bassa latenza nella distribuzione dei contenuti.

Amazon Cognito: Gestisce l'identità degli utenti (User Pool). Implementa i flussi di registrazione, verifica email (OTP) e login, fornendo una soluzione per le chiamate API verso il backend.

AWS Lambda: Rappresenta il cuore *Serverless* dell'applicazione. Gestisce la logica di business per la manipolazione dei voti e degli studenti, eseguendo il codice solo in risposta agli eventi HTTP, ottimizzando così i costi.

Amazon DynamoDB: Database NoSQL completamente gestito. Offre prestazioni a singola cifra di millisecondo per la memorizzazione di utenti, classi, voti e note, utilizzando un modello dati a tabella singola con chiavi primarie composite (PK/SK).

Amazon API Gateway: Funge da punto di ingresso unico per le API REST, gestendo il routing delle richieste verso le funzioni Lambda o i servizi containerizzati, e applicando policy di sicurezza (CORS).

Amazon ECS (Elastic Container Service) su Fargate: Gestisce l'orchestrazione del microservizio per le Note Disciplinari. L'utilizzo di AWS Fargate permette di eseguire container Docker senza la necessità di gestire o effettuare il provisioning dei server sottostanti (EC2).

Amazon ECR (Elastic Container Registry): Repository privato utilizzato per archiviare e versionare le immagini Docker del microservizio Python/Flask.

Amazon SNS (Simple Notification Service): Implementa il pattern di messaggistica *Publish/Subscribe*. È configurato con *Filter Policies* avanzate per garantire che le notifiche email relative ai voti raggiungano esclusivamente lo studente interessato, nel rispetto della privacy.

1.5 Metodologia DevOps e Infrastructure as Code

Un aspetto fondamentale del progetto è l'approccio all'automazione e alla gestione dell'infrastruttura.

1.5.1 Infrastructure as Code (IaC) con Terraform

L'intero ambiente cloud non è stato configurato manualmente tramite console, bensì definito in modo dichiarativo utilizzando **Terraform**. Questo approccio garantisce la riproducibilità dell'infrastruttura, il versionamento delle configurazioni e la possibilità di effettuare il provisioning o il *teardown* dell'intero ambiente con singoli comandi.

1.5.2 CI/CD con GitHub Actions

È stata implementata una pipeline di *Continuous Integration / Continuous Deployment* per automatizzare il rilascio del software. Ad ogni aggiornamento del codice sorgente sul repository, la pipeline si occupa di:

1. Costruire la nuova immagine Docker per il microservizio delle note.
2. Effettuare il push dell'immagine su Amazon ECR.
3. Aggiornare il servizio ECS per utilizzare la nuova versione del software senza dis-servizi.

Capitolo 2

Demo del progetto

In questo capitolo viene presentato il funzionamento dell'applicazione CloudRegistry attraverso una simulazione d'uso reale. Vengono illustrati i flussi principali: registrazione, gestione didattica lato docente e consultazione lato studente.

2.1 Accesso e Registrazione

All'apertura dell'applicazione, l'utente si trova di fronte alla schermata di Login. Se non possiede un account, può accedere alla procedura di registrazione.

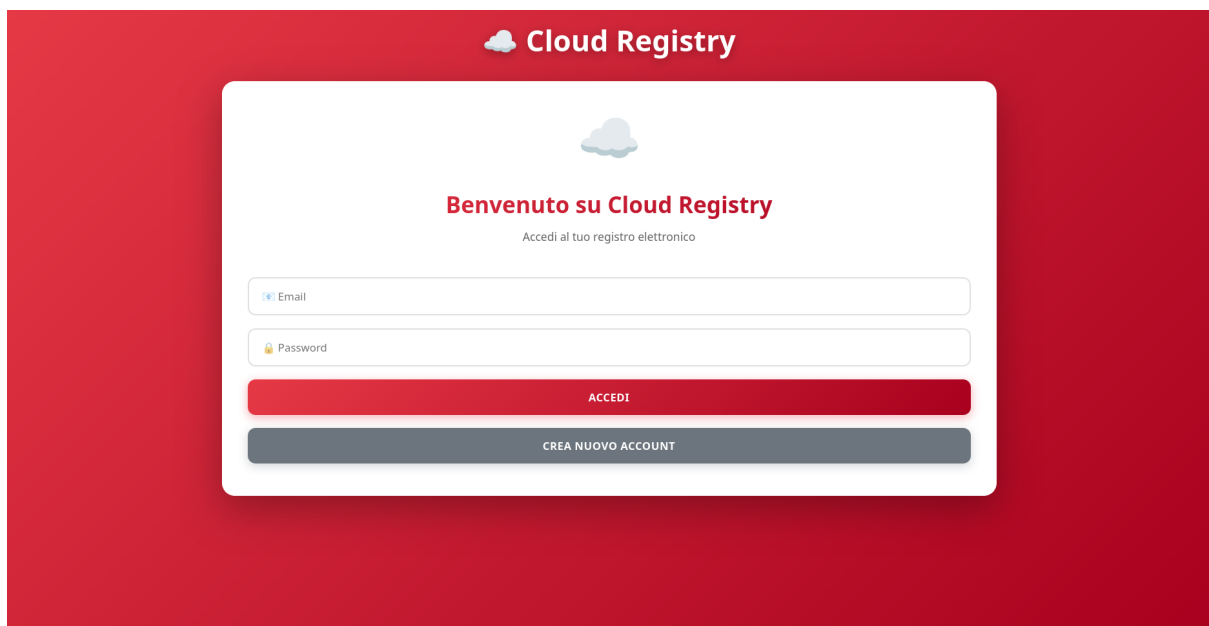
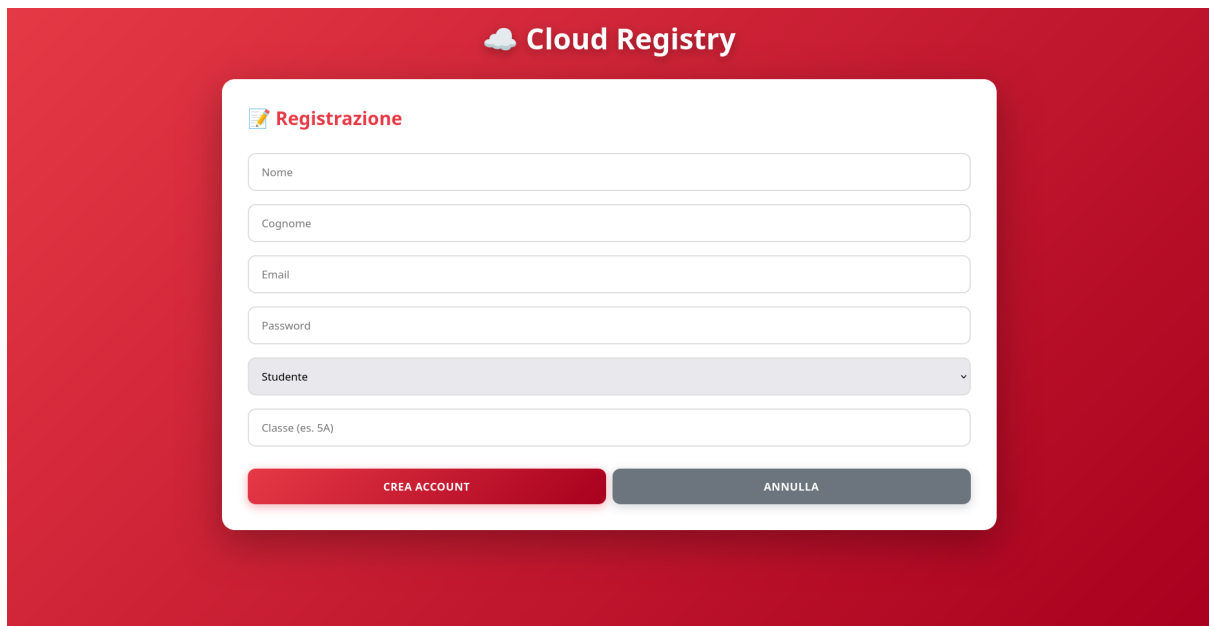


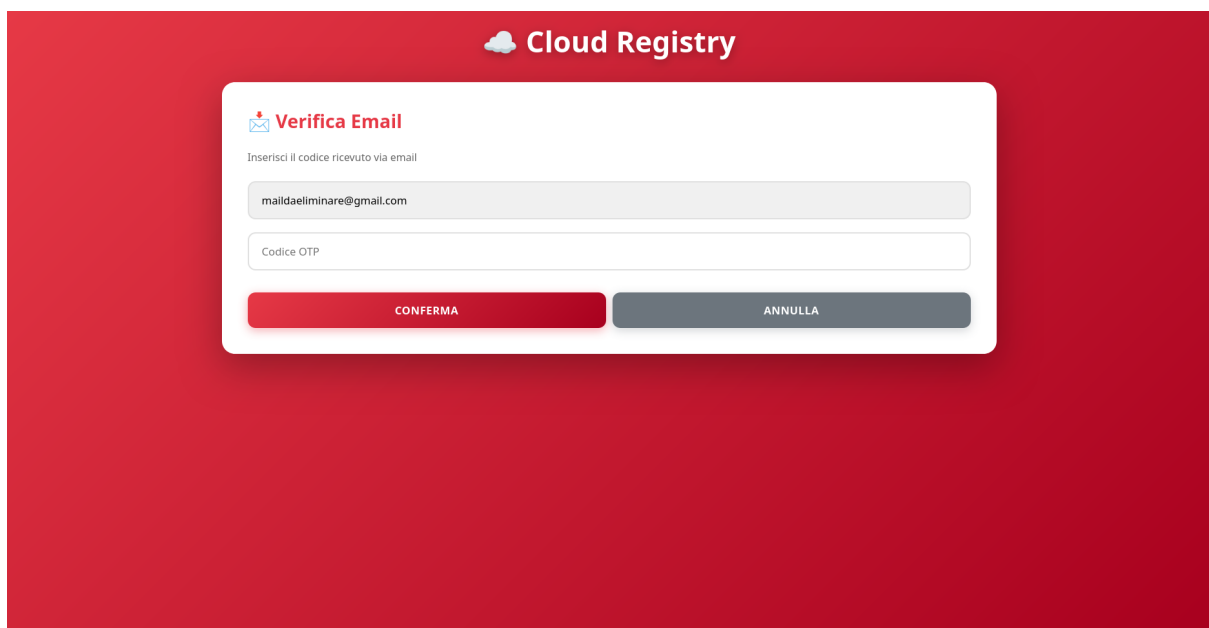
Figura 2.1: Schermata di Login dell'applicazione.

Durante la registrazione, l'utente deve specificare il proprio ruolo (Docente o Studente). Il sistema richiede una verifica dell'indirizzo email tramite un codice OTP (One-Time Password) inviato da Amazon Cognito per garantire l'autenticità dell'utente.



The screenshot shows the 'Cloud Registry' registration interface. At the top, there's a cloud icon and the text 'Cloud Registry'. Below this is a white card titled 'Registrazione' with a pencil icon. The card contains several input fields: 'Nome', 'Cognome', 'Email', 'Password', a dropdown menu for 'Studente' (with a downward arrow), and 'Classe (es. 5A)'. At the bottom of the card are two buttons: a red 'CREA ACCOUNT' button and a grey 'ANNULLA' button.

Figura 2.2: Modulo di registrazione e selezione del ruolo.



The screenshot shows the 'Cloud Registry' email verification interface. At the top, there's a cloud icon and the text 'Cloud Registry'. Below this is a white card titled 'Verifica Email' with an envelope icon. The card contains the text 'Inserisci il codice ricevuto via email' and a text input field with the email address 'maildaeliminare@gmail.com'. Below this is another input field labeled 'Codice OTP'. At the bottom of the card are two buttons: a red 'CONFERMA' button and a grey 'ANNULLA' button.

Figura 2.3: Inserimento codice OTP via interfaccia web

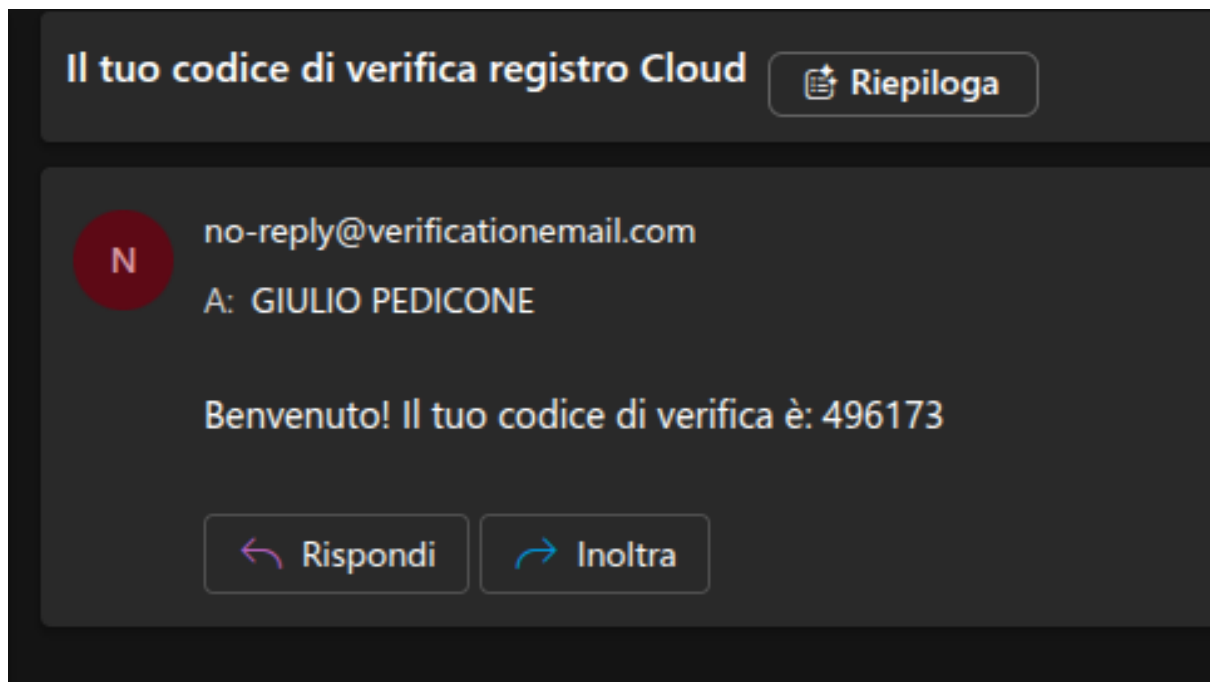


Figura 2.4: Mail ricevuta tramite codice OTP

2.2 Interfaccia Docente

Una volta effettuato l'accesso come Docente, viene mostrata la Dashboard di gestione. Qui il docente può filtrare gli studenti per classe e visualizzare l'elenco degli iscritti.

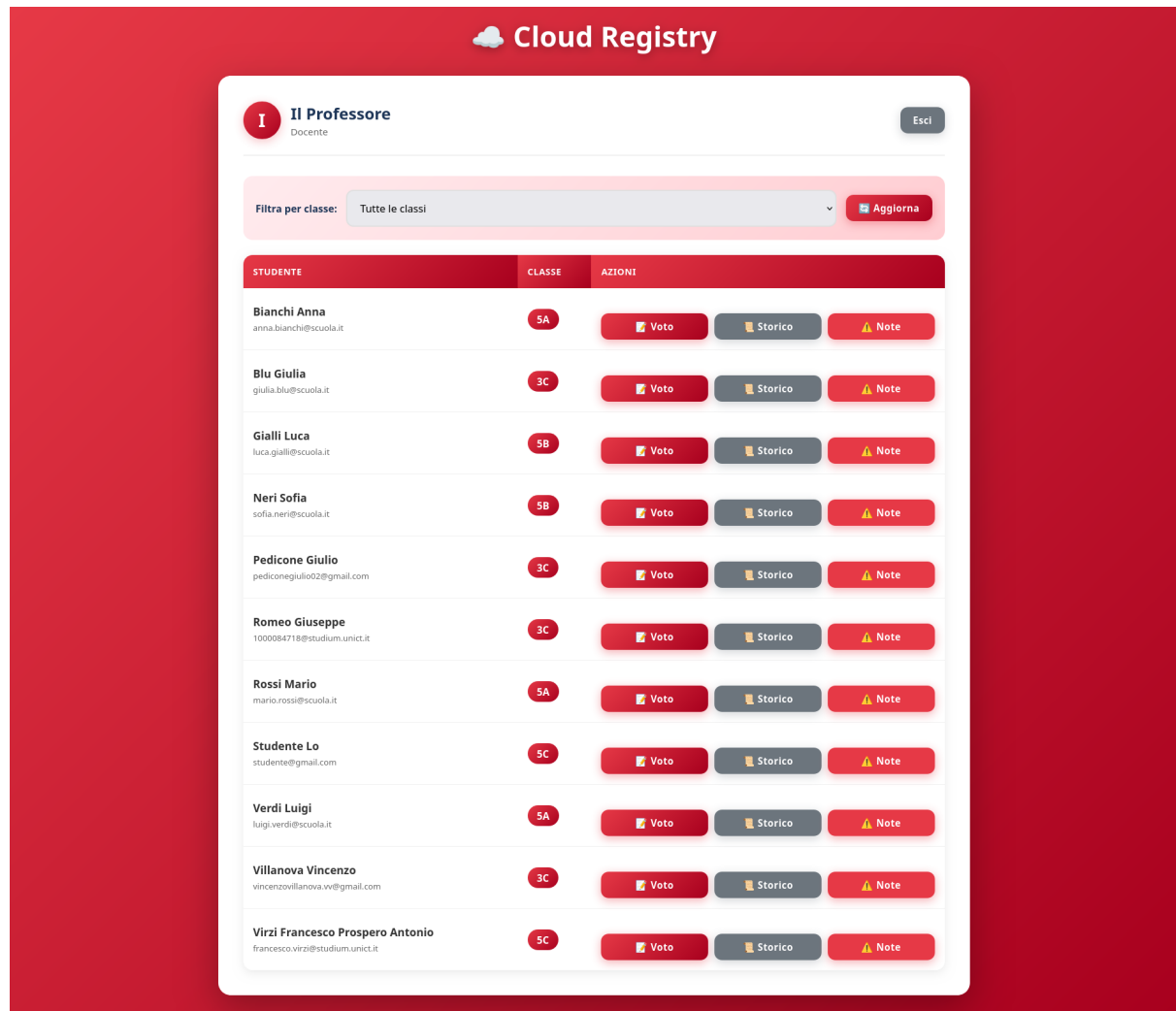


Figura 2.5: Dashboard del Docente con lista studenti e filtri per classe.

2.2.1 Assegnazione Voti e Notifiche

Cliccando sul pulsante "Voto", il docente apre un modale per l'inserimento della valutazione. Al momento della conferma, il sistema esegue due operazioni parallele:

1. Salva il dato su DynamoDB tramite Lambda.
2. Attiva il topic SNS per inviare una notifica email allo studente.

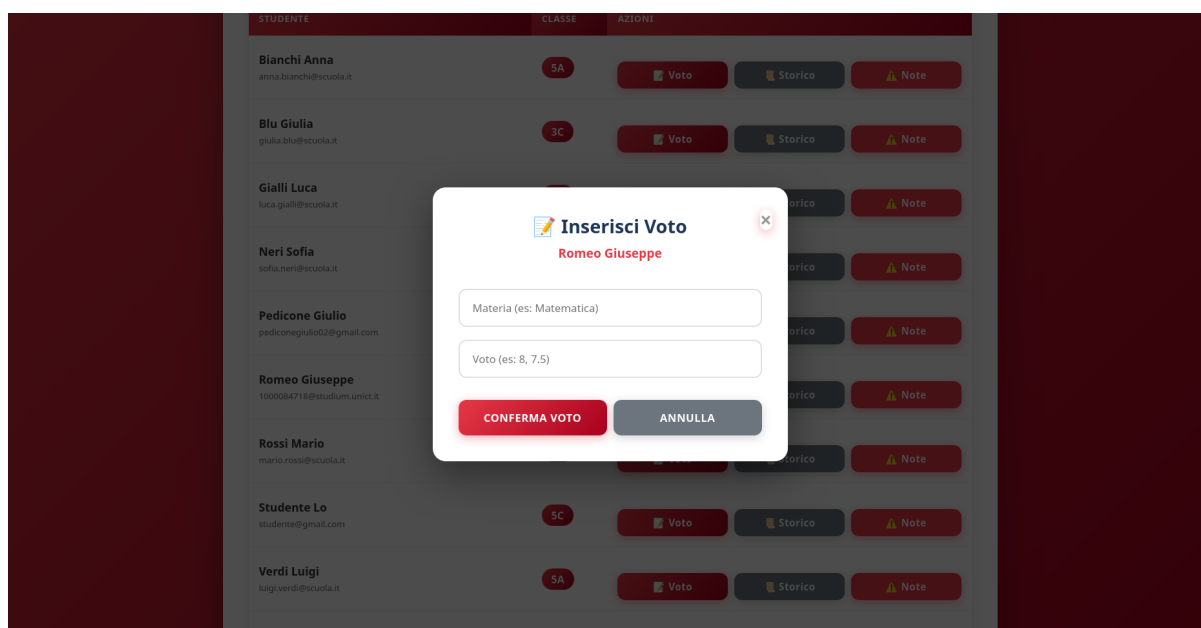


Figura 2.6: Inserimento di un nuovo voto



Figura 2.7: Esempio di email di notifica ricevuta dallo studente in tempo reale.

2.2.2 Gestione Note Disciplinari (Container)

La sezione Note è gestita dal microservizio su ECS Fargate. L'interfaccia permette di inserire note comportamentali o eliminare quelle esistenti tramite l'icona del cestino, funzionalità resa possibile dai permessi IAM granulari assegnati al task ECS.

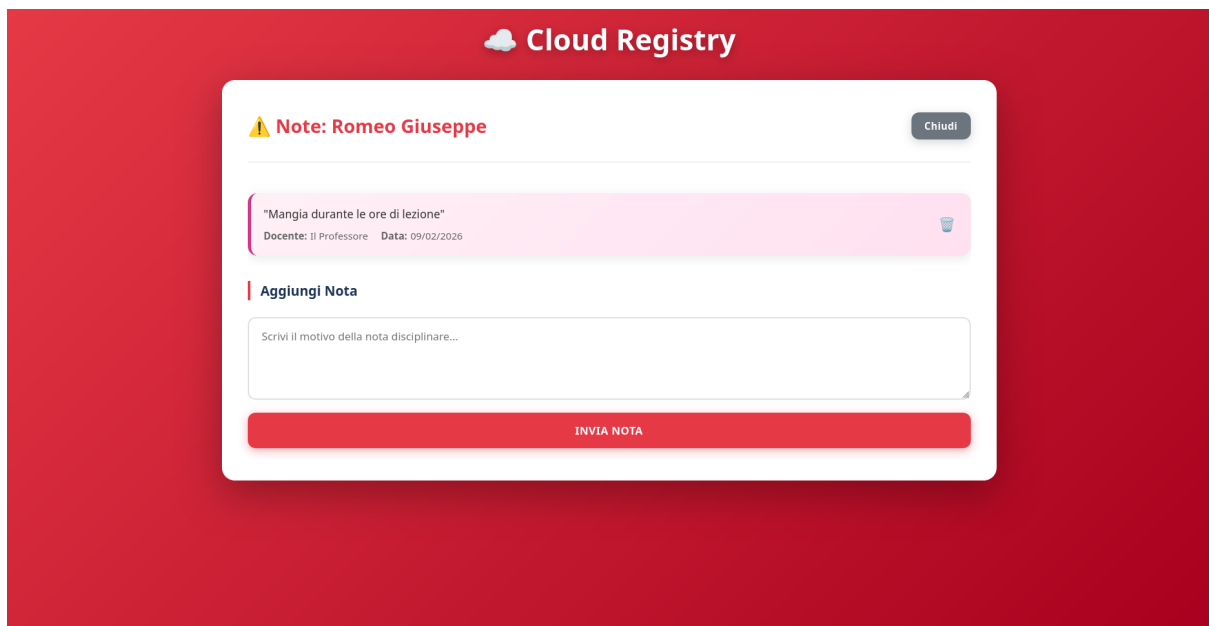


Figura 2.8: Interfaccia di gestione delle note disciplinari.

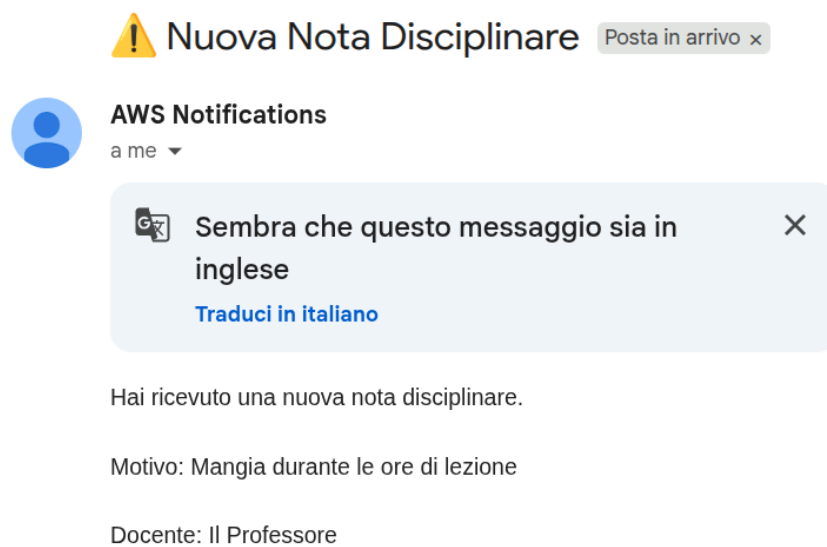


Figura 2.9: Esempio di email di notifica ricevuta dallo studente in tempo reale.

2.3 Interfaccia Studente

L'accesso studente offre una vista in sola lettura. La dashboard presenta immediatamente le medie calcolate per materia e lo storico completo dei voti.

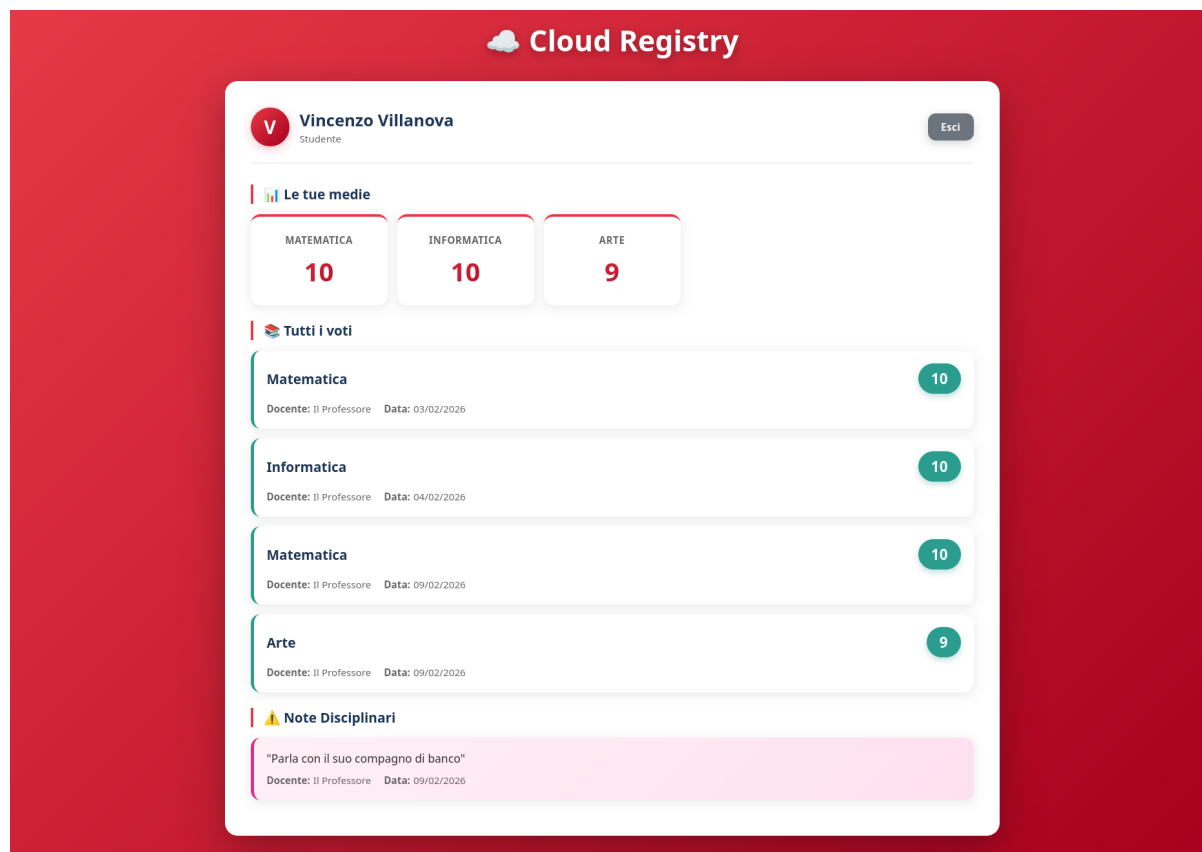


Figura 2.10: Dashboard dello studente con visualizzazione medie e storico voti.

Lo studente può visualizzare le note disciplinari a suo carico, ma, a differenza del docente, non dispone dei controlli per la cancellazione o la modifica, garantendo l'integrità dei dati disciplinari.

Capitolo 3

Dettagli Implementativi e Logica del Codice

In questo capitolo vengono analizzate le porzioni di codice più significative del progetto. L'obiettivo non è limitarsi alla descrizione delle funzioni standard, ma approfondire le soluzioni tecniche adottate per garantire automazione, scalabilità e tutela della privacy.

La struttura del repository è stata progettata per separare in modo chiaro i layer infrastrutturali, di backend e di frontend:

```
backend/                # Microservizio Python Flask (container ECS)
|-- app.py              # Logica applicativa
|-- Dockerfile          # Definizione dell'immagine
'-- requirements.txt
frontend/               # SPA (Single Page Application)
'-- index.html
index.py                # Funzione AWS Lambda (serverless)
main.tf                 # Infrastructure as Code (Terraform)
```

3.1 Automazione avanzata con Terraform

Uno degli aspetti più complessi nella gestione dell'Infrastructure as Code riguarda il deployment dei file statici. Definire manualmente una risorsa per ogni file HTML, CSS o JavaScript risulterebbe infatti poco scalabile e difficilmente manutenibile. Per questo motivo, nel file `main.tf` è stata implementata una logica di caricamento dinamico.

Utilizzando il costrutto `for_each` in combinazione con la funzione `fileset`, Terraform effettua automaticamente la scansione della directory `frontend/` e carica tutti i file presenti su Amazon S3. Inoltre, tramite la funzione `lookup`, viene assegnato dinamicamente il corretto *MIME type* (Content-Type) in base all'estensione del file, garantendo una corretta interpretazione delle risorse da parte del browser.

```

1 resource "aws_s3_object" "frontend_files" {
2   # 1. Scansione ricorsiva della cartella frontend
3   for_each = fileset("${path.module}/frontend", "**/*")
4
5   bucket = aws_s3_bucket.frontend.id
6   key    = each.value
7   source = "${path.module}/frontend/${each.value}"
8
9   # 2. Assegnazione dinamica del Content-Type
10  content_type = lookup({
11    "html" = "text/html",
12    "css"  = "text/css",
13    "js"   = "application/javascript",
14    "png"  = "image/png"
15  }, split(".", each.value)[length(split(".", each.value)) - 1],
16    ↪ "application/octet-stream")
17 }

```

Listing 3.1: Upload dinamico degli asset statici in `main.tf`

3.2 Logica serverless e privacy delle notifiche

Nel backend serverless (`index.py`), la principale criticità consisteva nell'implementazione di un sistema di notifica *publish/subscribe* tramite Amazon SNS che rispettasse la privacy degli studenti.

Un'eventuale iscrizione di tutti gli utenti a un unico topic generico avrebbe comportato la ricezione indiscriminata delle notifiche. Per evitare questo problema, è stata sfruttata la funzionalità di **Filter Policy** offerta da Amazon SNS.

Al momento della registrazione di un nuovo studente (azione `subscribe_student`), la funzione Lambda crea una sottoscrizione associata a una politica di filtro specifica: lo studente riceverà il messaggio esclusivamente se l'attributo `target_email` coincide con il proprio indirizzo email.

```

1 if action == 'subscribe_student':
2     student_email = body.get('email')
3
4     # Definizione della Filter Policy
5     filter_policy = {
6         "target_email": [student_email]
7     }
8
9     # Iscrizione al topic con vincolo di privacy
10    sns.subscribe(
11        TopicArn=SNS_TOPIC_ARN,
12        Protocol='email',
13        Endpoint=student_email,
14        Attributes={
15            'FilterPolicy': json.dumps(filter_policy)
16        }
17    )

```

Listing 3.2: Logica di iscrizione con Filter Policy in `index.py`

Di conseguenza, quando viene inserito un voto, il messaggio viene opportunamente “taggato” con l’email del destinatario, assicurando una consegna univoca e conforme ai requisiti di privacy.

3.3 Integrazione cloud nel microservizio containerizzato

Il microservizio dedicato alla gestione delle note disciplinari (backend/app.py) è implementato come applicazione Flask containerizzata. Un aspetto rilevante riguarda l’interazione del container, nonostante l’isolamento, con i servizi AWS gestiti.

Grazie ai ruoli IAM associati al task ECS e definiti tramite Terraform, il container eredita automaticamente i permessi necessari per scrivere su DynamoDB e pubblicare messaggi su SNS, senza ricorrere a credenziali statiche nel codice e senza l’uso di Access Key hardcoded.

Il frammento di codice seguente mostra come l’applicazione Flask, oltre a persistere il dato su un database NoSQL, invochi il servizio SNS per notificare lo studente dell’inserimento di una nuova nota disciplinare:

```
1 @app.route('/note', methods=['POST'])
2 def add_note():
3     # ... parsing dei dati ...
4
5     try:
6         # 1. Persistenza del dato su DynamoDB
7         table.put_item(Item=item)
8
9         # 2. Integrazione con SNS per la notifica
10        if SNS_TOPIC_ARN:
11            sns.publish(
12                TopicArn=SNS_TOPIC_ARN,
13                Message=f"Nueva nota disciplinare.\nMotivo: {testo}",
14                Subject="Nueva Nota",
15                MessageAttributes={
16                    'target_email': {
17                        'DataType': 'String',
18                        'StringValue': student_email
19                    }
20                }
21            )
22        return jsonify({"message": "Nota inserita"}), 201
23
24    except Exception as e:
25        return jsonify({"error": str(e)}), 500
```

Listing 3.3: Endpoint Flask per l’inserimento delle note e l’invio delle notifiche

Questa implementazione evidenzia la natura ibrida dell’architettura: due differenti compute engine, AWS Lambda e Amazon ECS Fargate, convergono sugli stessi servizi di storage (DynamoDB) e messaggistica (SNS), garantendo coerenza dei dati e un’esperienza utente unificata.

Capitolo 4

Automazione e Pipeline CI/CD

In un'architettura a microservizi distribuita come quella di CloudRegistry, il deployment manuale delle risorse (cosiddetto *ClickOps*) risulta non solo inefficiente, ma propenso all'errore umano. Per garantire affidabilità, riproducibilità e velocità di rilascio, è stata implementata una pipeline di **Continuous Integration e Continuous Deployment (CI/CD)** utilizzando **GitHub Actions**.

Il workflow è definito nel file `.github/workflows/deploy.yml` e viene innescato automaticamente ad ogni *push* sul ramo `main` del repository.

4.1 Struttura del Workflow

La pipeline è eseguita su un runner Linux (`ubuntu-latest`) e si articola in una serie di passaggi sequenziali che coprono l'intero ciclo di vita dell'infrastruttura e del software.

Di seguito vengono dettagliate le fasi critiche del processo di deployment.

4.1.1 Preparazione dell'Ambiente e Gestione dello Stato

Ogni esecuzione della pipeline avviene in un ambiente effimero. Il primo passo consiste nel recuperare il codice sorgente e preparare l'ambiente di lavoro per Terraform.

È stata implementata una strategia di "pulizia aggressiva" (*Clean Old State*) per rimuovere eventuali residui di configurazioni locali o file di stato corrotti che potrebbero interferire con il backend remoto S3 di Terraform.

```
1 - name: Checkout Code
2   uses: actions/checkout@v3
3
4 - name: Clean Old State
5   run: |
6     rm -rf .terraform
7     rm -f terraform.tfstate
8     rm -f terraform.tfstate.backup
```

Listing 4.1: Preparazione dell'ambiente di build

4.2 Deployment dell'Infrastruttura (IaC)

La gestione dell'infrastruttura è completamente automatizzata. Prima di applicare le modifiche, il codice della funzione Lambda (Python) viene compresso in un archivio .zip, requisito necessario per il deployment su AWS Lambda.

Successivamente, Terraform inizializza il backend remoto e applica le modifiche. L'uso del flag `-auto-approve` è essenziale in un ambiente non interattivo per confermare automaticamente il piano di esecuzione.

```
1 - name: Zip Backend Code
2   run: |
3     zip codice.zip index.py
4
5 - name: Terraform Apply
6   env:
7     AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
8     AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
9   run: |
10    terraform init -input=false
11    terraform apply -auto-approve -input=false
```

Listing 4.2: Packaging Serverless e Provisioning Terraform

4.3 Delivery del Microservizio Containerizzato

La parte più complessa della pipeline riguarda l'aggiornamento del microservizio "Note Disciplinari" basato su Docker. Poiché AWS Fargate non può "pullare" il codice direttamente da GitHub, è necessario costruire l'immagine container e caricarla su un registro privato (ECR).

La procedura si divide in tre step:

1. **Autenticazione:** Il runner ottiene un token di autorizzazione temporaneo per accedere ad Amazon Elastic Container Registry (ECR).
2. **Build & Push:** Viene costruita l'immagine Docker utilizzando il `Dockerfile` presente nella cartella `backend/` e successivamente caricata (push) sul repository ECR remoto con il tag `latest`.
3. **Deployment Forzato:** Infine, viene inviato un comando al cluster ECS per forzare un nuovo deployment del servizio. Questo obbliga i task Fargate a scaricare la nuova immagine appena caricata, garantendo che la versione in produzione sia sempre allineata con il repository Git.

```
1 - name: Login to Amazon ECR
2   id: login-ecr
3   uses: aws-actions/amazon-ecr-login@v1
4
5 - name: Build, tag, and push Docker image
6   env:
7     ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
8     ECR_REPOSITORY: registro-note-app
9     IMAGE_TAG: latest
10  run: |
11    cd backend
12    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
13    docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
14
15 - name: Force ECS Update
16  run: |
17    aws ecs update-service --cluster registro-cloud-cluster \
18      --service registro-note-service \
19      --force-new-deployment
```

Listing 4.3: Build Docker e aggiornamento ECS Fargate

4.4 Gestione della Sicurezza e dei Segreti

Un aspetto cruciale della pipeline è la gestione delle credenziali. Nessuna chiave di accesso AWS è hardcodata nel file YAML. Le credenziali sono archiviate in modo sicuro all'interno dei **GitHub Secrets** (`AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY`) e vengono iniettate come variabili d'ambiente solo durante l'esecuzione del job.

Questo approccio garantisce che il repository sia pubblico o condiviso senza rischio di esporre chiavi sensibili, rispettando le best practice di sicurezza *DevSecOps*.

Capitolo 5

Conclusioni e Sviluppi Futuri

Il progetto **CloudRegistry** ha dimostrato come l'adozione dei paradigmi *Cloud-Native* possa trasformare radicalmente lo sviluppo e la gestione di applicazioni web moderne. L'obiettivo principale non era solo la realizzazione di un registro elettronico funzionante, ma la creazione di un'architettura di riferimento che fosse resiliente, sicura e completamente automatizzata.

L'architettura ibrida realizzata, che combina la flessibilità del modello **Serverless** (AWS Lambda) con la portabilità dei **Microservizi Containerizzati** (Amazon ECS Fargate), ha permesso di ottimizzare sia i costi che le prestazioni:

- **Efficienza Operativa:** L'utilizzo di servizi gestiti come DynamoDB e Cognito ha abbattuto l'*overhead* di gestione, eliminando la necessità di amministrare server, database o sistemi di autenticazione proprietari.
- **Automazione Totale:** L'adozione di **Terraform** ha trasformato l'infrastruttura in codice versionabile. La gestione dinamica degli asset statici del frontend e la definizione dichiarativa delle risorse hanno reso il deployment ripetibile e privo di errori manuali (*ClickOps*).
- **Privacy by Design:** L'implementazione delle *Filter Policy* su Amazon SNS ha risolto una sfida critica nei sistemi a eventi: garantire che le notifiche raggiungano esclusivamente i destinatari corretti senza dover sviluppare logiche di routing complesse nel codice applicativo.

Un risultato significativo è stato raggiunto sul fronte della sicurezza. L'eliminazione di credenziali statiche (*hardcoded secrets*) a favore di ruoli IAM temporanei assunti a runtime dai container e dalle funzioni Lambda rappresenta una best practice fondamentale. Inoltre, la pipeline di CI/CD implementata con **GitHub Actions** ha dimostrato come sia possibile ridurre il *Time-to-Market*, automatizzando il ciclo di build, push e deploy dei microservizi Docker su AWS ECR ed ECS.

Per consultare il codice sorgente completo, la configurazione Terraform e la documentazione tecnica, si rimanda al repository ufficiale del progetto:

<https://github.com/giulioedicone02/CloudRegistry>